Cahier des Charges Fonctionnel & Technique : Application TaskMaster Pro

---

Objectif : Projet de validation pratique des fondations de JavaScript moderne (Vanilla JS / ES6+) avant la transition vers les frameworks front-end (React, Vue, Svelte).

> **Consigne pour l'agent de développement :** Implémenter l'application complète selon les spécifications fonctionnelles, l'arborescence des fichiers, les modèles de données et les règles strictes d'architecture ci-dessous, sans aucune bibliothèque tierce.

---

1. Arborescence & Structure des Fichiers

Le projet doit comporter exactement trois fichiers à la racine :

```
taskmaster/
├── index.html   (Structure sémantique et conteneurs d'ancrage)
├── style.css    (Feuille de styles moderne et classes d'états .completed et .active)
└── app.js       (Logique métier, état réactif en mémoire, délégation et persistance)
```

---

2. Principes d'Architecture & Contraintes Techniques

- ---

  Architecture guidée par l'état (State-Driven UI) : L'interface utilisateur est la projection stricte d'un état JavaScript en mémoire. Aucune donnée métier ne doit être extraite

directement du texte des balises HTML.

- **Immutabilité obligatoire :** Ne jamais muter l'état directement (bannir .push(), .splice()). Chaque modification d'état doit produire une nouvelle référence via le *spread operator* ..., .map() ou .filter().
- **Délégation d'événements (Event Delegation) :** Aucun écouteur d'événement individuel ne doit être posé sur les éléments de liste générés dynamiquement. Un unique écouteur doit être placé sur le conteneur parent (#task-list) pour intercepter les actions via e.target.matches() et e.target.closest().
- **Sécurité XSS :** Protéger l'insertion des textes saisis par l'utilisateur (utilisation de textContent ou échappement HTML strict).
- **Vanilla JS pur :** 100% JavaScript standard (ES6+), sans bibliothèque externe.

---

3. Modèle de Données (State Model)

Structure d'une tâche (Task Object)

```
{
  id: "1697382901234",      // Chaîne unique (ex: Date.now().toString())
  title: "Apprendre React", // Chaîne non vide issue de l'input utilisateur
  completed: false          // Booléen (statut d'achèvement)
}
```

Variables d'état principales

- tasks : Tableau contenant la liste des objets tâches (Array<Task>).
- currentFilter : Chaîne de caractères définissant le filtre actif :
  - "all" : afficher toutes les tâches.
  - "active" : afficher uniquement les tâches avec completed === false.

- "completed" : afficher uniquement les tâches avec completed === true.

---

4. Spécifications Fonctionnelles Détaillées

F1 : Création d'une tâche

- L'utilisateur saisit un libellé dans un champ <input> et soumet via le bouton ou la touche Entrée.
- L'événement submit du formulaire est intercepté avec e.preventDefault().
- Rejet de la soumission si le texte est vide après suppression des espaces superflus (.trim() === "").
- Ajout immuable de la nouvelle tâche dans l'état : tasks = [...tasks, newTask].
- Nettoyage du champ de saisie et remise du focus.
- Déclenchement automatique de la persistance et du réaffichage (render).

F2 : Modification du statut (Terminé / En cours)

- Au clic sur la case à cocher ou le libellé de la tâche, inversion de la valeur booléenne completed.
- Mise à jour immuable avec .map() :
  `tasks = tasks.map(task => task.id === targetId ? { ...task, completed: !task.completed } : task);`
- Application visuelle de la classe CSS .completed sur l'élément (texte barré, opacité réduite).

F3 : Suppression d'une tâche

- Au clic sur le bouton "Supprimer" associé à une tâche, retrait de la tâche via un filtrage immuable :
  `tasks = tasks.filter(task => task.id !== targetId);`

F4 : Filtrage des tâches

- Trois boutons de sélection de vue : **Toutes**, **En cours**, **Terminées**.
- Le clic met à jour currentFilter et active la classe CSS .active sur le bouton concerné.
- Le filtrage s'applique à la volée pendant le rendu avec .filter(), sans altérer le tableau source tasks.

F5 : Compteur dynamique

- Affichage en temps réel du nombre de tâches en cours restantes (tasks.filter(t => !t.completed).length).

F6 : Persistance dans le navigateur (LocalStorage)

- Clé de stockage dédiée : "taskmaster_tasks".
- Sauvegarde automatique à chaque mutation de l'état avec JSON.stringify().
- Initialisation au démarrage : lecture avec localStorage.getItem() et parsing avec JSON.parse(). En cas d'absence de données, repli sur un tableau vide [].

---

5. Fichiers et Codes de Référence Fournis

---

1. `index.html`

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TaskMaster Pro - JS Vanilla</title>
  <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
  <div class="app-container">
    <header class="app-header">
      <h1>TaskMaster Pro</h1>
      <p class="subtitle">Gestionnaire d'état réactif en Vanilla JS</p>
    </header>

    <form id="task-form">
      <input
        type="text"
        id="task-input"
        placeholder="Ajouter une tâche..."
        required
        autocomplete="off"
      >
      <button type="submit" id="add-btn">Ajouter</button>
    </form>

    <div class="filters" id="filter-container">
      <button class="filter-btn active" data-filter="all">Toutes</button>
      <button class="filter-btn" data-filter="active">En cours</button>
      <button class="filter-btn" data-filter="completed">Terminées</button>
    </div>

    <ul id="task-list"></ul>

    <footer class="footer-stats">
      <span id="task-count">0 tâche(s) en cours</span>
    </footer>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

2. `style.css`

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial,
sans-serif;
}

body {
  background-color: #f0f2f5;
  color: #1c1e21;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  min-height: 100vh;
  padding: 40px 16px;
}

.app-container {
  background: #ffffff;
  width: 100%;
  max-width: 520px;
  border-radius: 12px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
  padding: 28px;
}

.app-header {
  text-align: center;
  margin-bottom: 24px;
}

.app-header h1 {
  font-size: 26px;
  color: #1a73e8;
```

```
  font-weight: 700;
}

.app-header .subtitle {
  font-size: 13px;
  color: #65676b;
  margin-top: 4px;
}

#task-form {
  display: flex;
  gap: 8px;
  margin-bottom: 20px;
}

#task-input {
  flex: 1;
  padding: 12px 14px;
  border: 1px solid #ccd0d5;
  border-radius: 8px;
  font-size: 15px;
  outline: none;
  transition: border-color 0.2s;
}

#task-input:focus {
  border-color: #1a73e8;
}

#add-btn {
  background-color: #1a73e8;
  color: #fff;
  border: none;
  border-radius: 8px;
  padding: 12px 20px;
  font-size: 14px;
  font-weight: 600;
```

```
  cursor: pointer;
  transition: background-color 0.2s;
}

#add-btn:hover {
  background-color: #1557b0;
}

.filters {
  display: flex;
  gap: 8px;
  margin-bottom: 18px;
}

.filter-btn {
  flex: 1;
  background: #f0f2f5;
  border: none;
  padding: 8px;
  border-radius: 6px;
  font-size: 13px;
  font-weight: 500;
  color: #65676b;
  cursor: pointer;
  transition: all 0.2s;
}

.filter-btn:hover {
  background: #e4e6eb;
}

.filter-btn.active {
  background: #1a73e8;
  color: #ffffff;
}

#task-list {
```

```
  list-style: none;
  margin-bottom: 20px;
}

.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px;
  border-bottom: 1px solid #e4e6eb;
  transition: background-color 0.15s;
}

.task-item:hover {
  background-color: #f8f9fa;
}

.task-left {
  display: flex;
  align-items: center;
  gap: 12px;
  flex: 1;
}

.task-checkbox {
  width: 18px;
  height: 18px;
  cursor: pointer;
}

.task-title {
  font-size: 15px;
  word-break: break-all;
  cursor: pointer;
}

.task-item.completed .task-title {
```

```
  text-decoration: line-through;
  color: #8c939d;
}

.delete-btn {
  background: none;
  border: none;
  color: #dc3545;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  padding: 4px 8px;
  border-radius: 4px;
  transition: background-color 0.2s;
}

.delete-btn:hover {
  background-color: #fde8e8;
}

.footer-stats {
  padding-top: 14px;
  border-top: 1px solid #e4e6eb;
  font-size: 13px;
  color: #65676b;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

3. `app.js` (Structure attendue à implémenter)

```
/**
 * TaskMaster Pro - Application en JavaScript Moderne (Vanilla JS)
 * Architecture : State-Driven UI & Immutabilité
 */
```

```
// ==========================================
// 1. ÉTAT DE L'APPLICATION (STATE)
// ==========================================
const STORAGE_KEY = "taskmaster_tasks";
let tasks = [];
let currentFilter = "all";

// ==========================================
// 2. SÉLECTEURS DU DOM
// ==========================================
// Formulaire, input, conteneur liste, boutons filtres, compteur...

// ==========================================
// 3. FONCTIONS MÉTIER & PERSISTANCE
// ==========================================
// - loadTasks() : lecture du localStorage
// - saveTasks() : écriture dans le localStorage
// - addTask(title) : ajout immuable [...tasks, newTask]
// - toggleTask(id) : bascule complétée avec .map()
// - deleteTask(id) : suppression avec .filter()
// - setFilter(filter) : modification du filtre courant

// ==========================================
// 4. MOTEUR DE RENDU (RENDER)
// ==========================================
// - Filtrer les données selon currentFilter
// - Générer et insérer les éléments HTML
// - Mettre à jour le compteur en temps réel

// ==========================================
// 5. GESTION DES ÉVÉNEMENTS (EVENT LISTENERS)
// ==========================================
// - submit sur taskForm
// - délégation de clics sur taskList (toggle et delete)
// - clic sur les boutons de filtre
// - initialisation au chargement de la page (init)
```

---

6. Critères d'Acceptation & Validation Technique

- ---

  [ ] **Persistance totale :** Actualiser la page (F5) conserve toutes les tâches et leur état sans perte.
- [ ] **Zéro écouteur orphelin :** Aucun addEventListener n'est attaché individuellement aux <li> ou boutons de suppression. Tout passe par la délégation d'événements sur #task-list.
- [ ] **Sécurité XSS :** Une saisie comme <script>alert(1)</script> ou <img src="x" onerror="alert(1)"> est affichée comme simple texte et n'exécute aucun code.
- [ ] **Immutabilité stricte :** L'état tasks n'est jamais modifié par mutation directe (pas de .push() ni de modification d'attribut direct).